

Exercices — Gestion des Exceptions Java & Spring Boot

Niveau : après Spring Boot et l'architecture Controller → Service → Repository, avant JPA.

Objectif : comprendre les exceptions, créer des exceptions métier, les propager correctement et construire une API REST avec des erreurs propres.

Règle : fais d'abord fonctionner le cas normal, puis ajoute les erreurs progressivement. N'utilise pas encore JPA, Security ou les microservices.

Progression

#	Exercice	Niveau	Compétence
1	Fondamentaux Java	■	throw, throws, try/catch/finally
2	Propagation	■	Comprendre la remontée d'une exception
3	Exceptions personnalisées	■■	Créer des RuntimeException métier
4	API REST	■■	Exceptions dans Controller/Service
5	@ExceptionHandler	■■	Exception → HTTP status
6	@RestControllerAdvice	■■■	Centraliser les erreurs
7	ErrorResponse	■■■	Standardiser le JSON
8	Validation	■■■	Gérer les erreurs de validation
9	Tests	■■■	JUnit 5 + Mockito
10	Projet final	■■■■	API CRUD robuste

Exercice 1 — Les fondamentaux Java

Objectif : Maîtriser throw, throws et try/catch/finally.

Travail demandé :

- Créer **divide(int a, int b)** et lancer une exception si $b = 0$.
- Créer **readAge(int age)** et refuser un âge négatif.
- Tester les méthodes avec try/catch/finally.
- Créer un exemple avec throws et observer la propagation.

Contraintes :

- Pas de Spring Boot.
- Pas de classe complexe.

Résultat attendu : Tu dois pouvoir expliquer la différence entre **throw** (lancer) et **throws** (déclarer/propager).

Exercice 2 — Comprendre la propagation

Objectif : Voir comment une exception remonte de méthode en méthode.

Travail demandé :

- Créer trois classes simulant Repository → Service → Controller.
- Le Repository lève une exception si un objet n'existe pas.
- Le Service appelle le Repository sans masquer l'exception.

- Le Controller intercepte l'exception.
- Ajouter des messages/logs pour visualiser le trajet.

Contraintes :

- Pas de `@ExceptionHandler`.
- Pas de `@RestControllerAdvice`.

Résultat attendu : Tu dois pouvoir dessiner le trajet de l'exception et expliquer où elle est finalement traitée.

Exercice 3 — Exceptions personnalisées

Objectif : Comprendre pourquoi on crée des exceptions métier.

Travail demandé :

- Créer **ProductNotFoundException**.
- Créer **DuplicateProductException**.
- Créer **InvalidProductPriceException**.
- Les faire hériter de `RuntimeException`.
- Dans `ProductService`, lancer l'exception correspondant à chaque règle.

Contraintes :

- Messages explicites.
- Éviter d'utiliser `Exception` comme exception fourre-tout.

Résultat attendu : Un produit inexistant déclenche `ProductNotFoundException` avec un message utile.

Exercice 4 — Exceptions dans une API REST

Objectif : Intégrer les exceptions dans ton architecture `Controller` → `Service` → `Repository`.

Travail demandé :

- Créer un CRUD `Product` en mémoire.
- `GET /products/{id}` → exception si le produit n'existe pas.
- `POST /products` → refuser un doublon.
- `POST/PUT` → refuser un prix inférieur ou égal à zéro.
- Tester avec Postman ou curl.

Contraintes :

- Toujours respecter `Controller` → `Service` → `Repository`.
- Ne pas centraliser les erreurs pour l'instant.

Résultat attendu : L'API fonctionne et les erreurs sont bien déclenchées, même si leur présentation n'est pas encore parfaite.

Exercice 5 — `@ExceptionHandler`

Objectif : Transformer une exception en réponse HTTP.

Travail demandé :

- Ajouter `@ExceptionHandler(ProductNotFoundException.class)` dans le `Controller`.
- Retourner HTTP 404.
- `DuplicateProductException` → 409.
- `InvalidProductPriceException` → 400.

- Comparer le comportement avant/après le handler.

Contraintes :

- Un handler doit correspondre à une exception précise.

Résultat attendu : Un produit inexistant retourne 404 Not Found.

Exercice 6 — @RestControllerAdvice

Objectif : Centraliser les handlers afin de ne pas répéter le même code dans tous les Controllers.

Travail demandé :

- Créer GlobalExceptionHandler avec @RestControllerAdvice.
- Déplacer les handlers du ProductController vers cette classe.
- Ajouter un CustomerController.
- Vérifier que CustomerController bénéficie aussi du traitement global.

Contraintes :

- Les Controllers ne doivent plus contenir la gestion globale des exceptions.

Résultat attendu : Un seul composant central gère les erreurs communes de l'API.

Exercice 7 — Créer un ErrorResponse

Objectif : Avoir un format JSON cohérent pour toutes les erreurs.

Travail demandé :

- Créer un record ErrorResponse avec au minimum status, message et path.
- Faire retourner ce record par GlobalExceptionHandler.
- Ajouter éventuellement timestamp et errorCode.
- Tester 400, 404 et 409.

Contraintes :

- Ne jamais exposer une stack trace au client.
- Garder une structure cohérente.

Résultat attendu : Par exemple :

```
{
  "status": 404,
  "message": "Product not found",
  "path": "/products/123"
}
```

Exercice 8 — Validation des données

Objectif : Gérer les données invalides envoyées par le client.

Travail demandé :

- Créer ProductRequest avec @NotBlank sur name.
- Ajouter @Positive sur price.
- Utiliser @Valid dans le Controller.
- Déclencher plusieurs erreurs volontairement.
- Gérer MethodArgumentNotValidException dans GlobalExceptionHandler.
- Retourner les champs invalides et leurs messages.

Contraintes :

- Séparer validation et erreur métier.
- Le client doit savoir quel champ corriger.

Résultat attendu : Une requête invalide retourne une erreur structurée avec les champs concernés.

Exercice 9 — Tester les exceptions

Objectif : Vérifier automatiquement que les exceptions sont réellement lancées.

Travail demandé :

- Tester `findById()` avec un ID inexistant.
- Utiliser `assertThrows`.
- Vérifier le type d'exception et, si pertinent, le message.
- Avec Mockito, vérifier l'appel au Repository.
- Ajouter les tests pour doublon et prix invalide.

Contraintes :

- Tester le comportement observable.
- Éviter les tests dépendant inutilement de l'implémentation.

Résultat attendu : Le test échoue si le Service ne lève plus l'exception attendue.

Exercice 10 — Projet final : Product API robuste

Objectif : réunir toutes les notions dans une vraie petite API Spring Boot.

Fonctionnalités :

- Product : id, name, description, price.
- GET /products
- GET /products/{id}
- POST /products
- PUT /products/{id}
- DELETE /products/{id}
- Recherche par nom.

Exceptions :

- ProductNotFoundException → 404 Not Found.
- DuplicateProductException → 409 Conflict.
- InvalidProductPriceException → 400 Bad Request.
- Exception inattendue → 500 Internal Server Error.

Architecture :

- Controller → Service → Repository.
- GlobalExceptionHandler avec @RestControllerAdvice.
- ErrorResponse standard.
- Validation avec @Valid.
- Aucune stack trace exposée au client.

Tests minimum :

- Produit trouvé.
- Produit inexistant.
- Création réussie.
- Doublet.
- Prix invalide.
- Validation invalide.
- Modification d'un produit inexistant.
- Suppression d'un produit inexistant.

Défi supplémentaire

Ajoute un second domaine, par exemple Customer. Vérifie que GlobalExceptionHandler reste générique, tout en gardant les exceptions métier dans leur domaine. Tu peux aussi introduire des codes stables comme **PRODUCT_NOT_FOUND** ou **PRODUCT_ALREADY_EXISTS**.

Checklist avant de passer à JPA

- Expliquer checked vs unchecked exceptions.
- Expliquer throw vs throws.
- Créer une RuntimeException personnalisée.

- Comprendre la propagation.
- Associer une erreur à un HTTP status adapté.
- Utiliser `@ExceptionHandler`.
- Utiliser `@RestControllerAdvice`.
- Créer `ErrorResponse`.
- Gérer `MethodArgumentNotValidException`.
- Tester les exceptions avec JUnit 5 + Mockito.
- Distinguer erreur métier et erreur technique.

Planning conseillé — 7 jours

- Jour 1 : exercices 1–2.
- Jour 2 : exercice 3.
- Jour 3 : exercices 4–5.
- Jour 4 : exercices 6–7.
- Jour 5 : exercice 8.
- Jour 6 : exercice 9.
- Jour 7 : exercice 10.

Conseil important : après chaque exercice, explique à voix haute : « Qui crée l'exception ? Qui la reçoit ? Qui la traite ? Quelle réponse HTTP le client reçoit ? » Si tu peux expliquer ce trajet sans regarder le code, la notion est en train d'être assimilée.